

進捗報告

下沢 亮太郎

2026年10月07日

1 先週の実施予定内容

- Gemma 4 12B を土台に検証済みデータで SFT し、4B と同じ評価集合で比べる
- 雛形外への汎化を伸ばすため、雛形を追加して学習データの種別を広げる
- GEPA の反省プロンプトを「一般原則を優先」に差し替えた条件を試す
- 問題集が簡単すぎるという指摘（前回 Q14）に対し、厳密解が出ない規模の問題を混ぜる

2 実施した内容

2.1 概要

目標は「要件を入れると最適化プログラムを書く、最適化専用の小型エージェント」で、柱は GEPA によるプロンプト最適化と、プロンプト最適化と fine-tuning を交互に回すメタ最適化（DSPy の BetterTogether）である。今回の結果は次の四点に要約できる。

1. 厳密解が出ない規模の「大規模問題」20 種別を導入し、学習用・評価用に instance を分けた。素の Gemma 4 12B はこの評価 120 問を 1 問も解けない。
2. 検証済みの正解コードで Gemma 4 12B を学習すると、大規模評価 120 問で LoRA が 88 問、全層学習が 93 問を正解した。同じデータで 4B を学習すると 81 問で、12B の 9 割強を 1/3 の大きさで出せる。
3. BetterTogether（プロンプト → 重み → プロンプト）を、学習済みモデルからと素のモデルからの 2 通りで回した。素のモデルでは GEPA が指示文を動かし、雛形テストで +19 問（345→364）効いた。しかし学習済みモデルは学習時の指示文に張り付いており、学習後の GEPA は 3 回で 52 案を試して 1 案も採用できなかった。
4. 効果は学習で見た種別に限られる。学習データのなかった 3 種別はテスト 18 問中 0 問で、雛形外 11 問は学習を重ねるほど減った（素 3→学習後 2→自己蒸留後 1）。当面は対象を学習した種別に絞り、Gemma 4 をその専用エンジンとして仕上げる。

2.2 前回 (9/9) からやったこと

- 雛形を 28 問から 89 問に増やし、検証済みの学習データを 6,619 対から 15,088 対にした。
- 新たに受け取った大規模問題 28 問を取り込み、検証器 20 種別を書き、Qwen3.6 / 3.8, Gemma, Fable で測った (Fable 22 問, Qwen3.8 11 問, Gemma 5 問)。実行上限を 1,800 秒に伸ばし、違反を見せて書き直させる修復ループを足した。
- 大規模問題 20 種別 $\times$ 40 instance (計 800) を生成し、学習用 600 / 検証用 80 / テスト用 120 に分けた。参照解は既存の教師コード 95 本の最良値。
- Opus 5.5 に大規模問題 100 問を「書いて試して直す」形で解かせ、正解 95 問を他の instance にも再生して 630 対の学習データにした。GLM-5.3-Flash は 1 問 40 分かかり正解 0 で不採用。
- 雛形 15,088 対と大規模 630 対を合わせて Gemma 4 12B を LoRA と全層学習で学習した。全層学習は bf16 のまま更新した 1 回目が失敗し、fp32 主重みに直して成功。大規模テスト 120 問で LoRA 88, 全層 93 (素の Gemma は 0)。
- 同じデータで Agents-A1-4B を LoRA 学習し、大規模 81 問。12B の 9 割強を 1/3 の大きさで出せた。
- 交互最適化 (BetterTogether, $p \rightarrow w \rightarrow p$) を 2 周回した。学習済みモデルからの GEPA は 51 案中採用 0。素のモデルから始めると GEPA で指示文が動き (雛形 345 $\rightarrow$ 364), その指示文で学習し直しても S0 と差がなく、学習後の GEPA はまた採用 0。
- GEPA の反省プロンプトを「一般則だけ、約 4,000 字以内」に差し替え、学習済みモデルを DSPy から学習時と同じ形式で呼ぶ部品を作った。GEPA を GPU 1 枚で回せるようにした。
- 素の Gemma 向けに進化させた指示文を Qwen3.6 に転用し、大規模テストの正解が 4 $\rightarrow$ 9 (有意ではない)。
- 学習データが 0 件だった 3 種別を直した。乗務員ペアリングは検証器を 302 系 / 312 系で分けて参照解を付け直し、ポートフォリオは問題文の行列の書き出しをやめて 4.5 万字 $\rightarrow$ 8.6 千字にした。20 種別すべてに学習データが入り、4B を学習し直している。
- 今後の計画を 4 者 (Codex Sol / Luna, Claude Fable, 実装担当) に出させて突き合わせ、report/20261007/next_steps_review.md にまとめた。
- 報告書 (RESCORE_REPORT.md 23~32 章) と手順書 (OPERATIONS.md 7~13 章), 月次の質疑応答 (qa_answers.md), Gemma の LoRA / FFT の質疑応答 (qa_gemma4_ft.md) を書いた。

2.3 学習データとテストデータ

学習とテストに使った問題は、出どころの違う4つの集合に分けられる（表1）. どの集合も「問題文」と「instance（数値データ）」と「参照解」からなる. モデルには参照値を伏せた問題文だけを渡し、返ってきた `solve()` を instance で実行して検証器で採点する.

表 1: 問題の集合と役割

集合	中身	規模	参照解	役割
雛形問題	同梱 100 問のうち厳密に解ける 89 問を雛形にし、数値を変えた instance を生成	数個～数十個の要素	厳密ソルバー（CP-SAT, LP, 全列挙）の最適値	学習・テスト
大規模問題	新たに受け取った 28 問を元に、20 種別ごとに同じ分布の instance を生成	数百～数千の要素. 厳密解は出ない	既存の教師コード 95 本を走らせた中の最良値（最適性は未証明）	学習・テスト
大規模の元問題	受け取った 28 問そのもの（prob_301～330）	同上	同梱の参照解	テストのみ
雛形外	同梱 100 問のうち雛形にできなかった 11 問	小規模	同梱の参照解	テストのみ（汎化の測定）

雛形問題と大規模問題は、1つの種別から乱数の種を変えて多数の instance を作り、学習用・検証用・テスト用に分けている（表2）. テスト用の instance は学習に一度も使っていない. ただし種別は学習と共通なので、雛形テストと大規模テストが測るのは「学習で見た種類の、新しい数値の問題を解けるか」である. 学習で見えていない種類への汎化は、雛形外 11 問で測る.

表 2: 分割（instance 数）

集合	学習用	検証用	テスト用	分け方
雛形問題（89 雛形）	3,560（各 40）	356（各 4）	445（各 5）	乱数の種を分ける
大規模問題（20 種別）	600（各 30）	80（各 4）	120（各 6）	種別ごとに 800 を生成して分割
大規模の元問題	—	—	28	—
雛形外	—	—	11	—

学習データは「問題文 → 正解コード」の対で、コードは必ず学習用 instance で実際に実行し、検証器で正解を確かめたものだけを使った（表3）. 正解の条件は、制約違反が0で、参照解からの差が+10%以内であること. モデルの出力を確かめずに学習に入れたものはない.

表 3: 学習データの中身

出典	作り方	学習対
雛形問題の教師コード	Fable・Qwen・Gemma・Minstral が同梱問題に書いた正解コードと手書き 11 本, 計 412 本を学習用 instance で実行し, 正解した対だけ残す (16,480 対中 15,088 対)	15,088
大規模問題の Opus 5.5 の解答	学習用 600 問から種別ごとに 5 問, 計 100 問を Opus 5.5 に「書いて試して直す」形で解かせ, 正解 95 問のコード (中身は 21 種類) を他の学習用 instance でも実行して正解したもの	630
合計 (学習に使った形)	大規模問題の 630 対は件数が少ないので 2 回ずつ入れた. 最大長 16,384 token を超えるポートフォリオ 2 種別の 120 行は落ちた	16,228

図 1 に, 問題の集合から学習データとテストデータが作られる流れを示す.

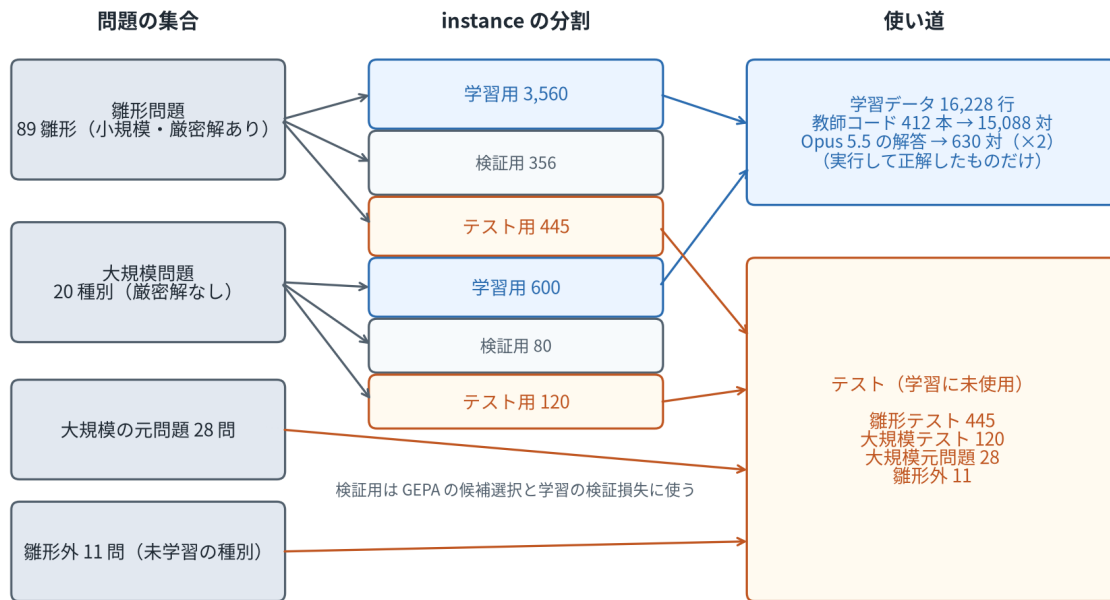


図 1: 学習データとテストデータの作り方 (青が学習, 橙がテスト)

学習データに入っていない種別が大規模問題に 3 つあった. ポートフォリオ 2 種別は, 問題文の構造要約が収益率の行列 (300×1,200) をそのまま書き出して 4 万 token 近くになり, 学習の最大長を超えていた. 乗務員ペアリングは, 元問題 2 つで規則が違う (各便ちょうど 1 回か, 1 回以上か) のに検証器が両方に重複を許していたため, 問題文どおりに解いた Opus の解が参照解より悪いと判定され, 学習データが 0 件になった. この 3 種別 (テスト 18 問) は, 表 4 の学習済みモデルにとって実質的に「見ていない種別」である.

いずれも直した. 行列は形と先頭の数値だけ書くようにして問題文を 8.6 千字に収め, 乗務員ペアリングは検証器を規則ごとに分けて参照解を付け直した. 20 種別すべてに学習データが入った 645 対で, Gemma 4 を学習し直す (結果は次回).

2.4 大規模問題での特化学習と小型化

表 3 の 16,228 行で Gemma 4 12B を学習した (表 4). LoRA は全線形層に rank 32 の adapter を足して H200 1 枚で 5 時間 38 分, 全層学習は 120 億パラメータすべてを更新して H200 2 枚で 6 時間 7 分である. 同じデータで Agents-A1-4B も LoRA で学習した (3 時間 39 分). いずれも 1 epoch, 思考を書かずに直接コードを出す形式である.

表 4: 正解数 (違反 0・参照解から+10%以内)

モデル	雛形テスト 445	大規模テスト 120	大規模元問題 28	雛形外 11
素の Gemma 4 12B (思考なし)	345	0	3	3
Qwen3.6 27B (思考あり)	—	4	—	—
Gemma 4 12B LoRA	442	88	12	2
Gemma 4 12B 全層学習	445	93	14	—
Agents-A1-4B LoRA	440	81	10	3

注: 乗務員ペアリングの検証器と参照解を評価後に直したため, 素の Gemma 4 12B の大規模テストは 1 問が正解に変わる (0→1). 他の条件の数字は変わらない.

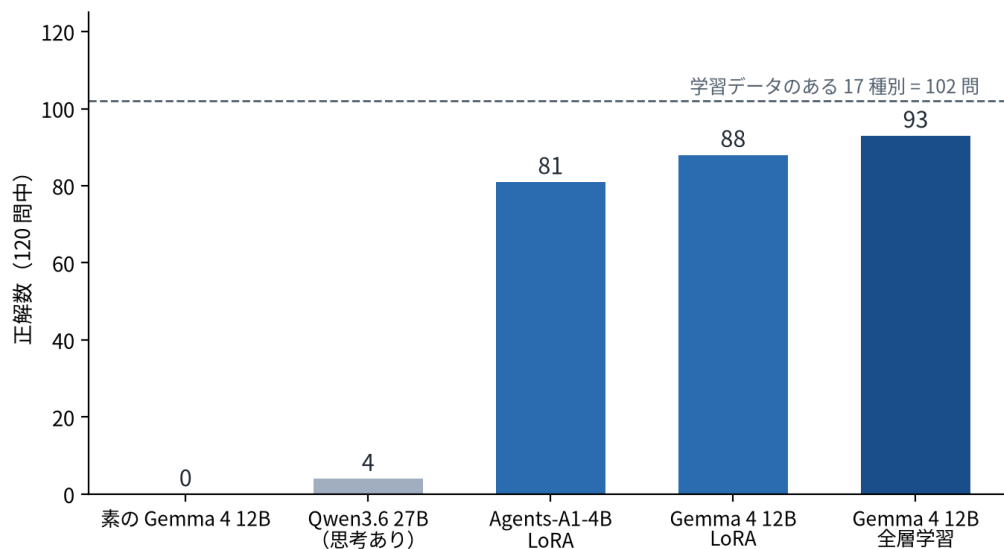


図 2: 大規模テスト 120 問の正解数 (破線は学習データのある 17 種別の問題数)

大規模テストで学習後のモデルが外した問題の大半は, 学習データに入っていなかった 3 種別 (18 問) である (図 2). 学習済みの 17 種別に限ると, 全層学習は 102 問中 93 問, LoRA は 88 問, 4B は 81 問を正解した. 全層学習の正解のうち 81 問は, 参照解 (主に Claude Fable が 10 分かけて出した解) と同等以上だった. LoRA と全層学習の差は問題単位で 8 対 3 (大規模テスト) で, 統計的に有意ではない.

全層学習の 1 回目は, 重みを bf16 のまま更新したために失敗した (大規模テスト 0 問). 学習率 $1e-5$ の更新が bf16 の丸め幅より小さく, 大半が消えていた. 重みを fp32 で持ち, 計算だけ bf16 で行う混合精度に直すと, 検証損失が 0.0305 から 0.0023 に下がり, 上記の結果になった.

2.5 交互最適化 (BetterTogether)

DSPy の BetterTogether(p=GEPA, w=BootstrapFinetune, strategy="p -> w -> p") と同じ順序で回した。プロンプト側は GEPA (反省は Qwen3.8, 一般則だけを書かせる反省テンプレート), 重み側は「その時点の指示文で解いて正解したものを教師データに足し, 学習し直す」である。出発点を変えて 2 周回した。

表 5: 学習済みモデルから始めた 1 周

段	内容	結果
p (GEPA)	S0 の指示文を進化 (6 時間 51 分)	46 案を試して採用 0. 指示文は変わらず
w(自己蒸留)	S0 で大規模の学習用 600 問を解き, 正解した 452 問を学習データに足して学習し直す (S1)	雛形 442→445, 大規模 88→91, 雛形外 2→1
p (GEPA)	S1 の指示文を進化 (6 時間 29 分)	5 案が全評価に進んだが, どれも元より低い (1.28 に対し 0.56~1.03). 採用 0

1 周目: 学習済みモデル (表 4 の LoRA, S0) から 学習済みのモデルは学習時の指示文に張り付いていて, 書き換えると出力の打ち切りや構文の崩れで壊れる。このためプロンプト側は動かず, 1 周は実質「自分の正解で学び直す自己蒸留」になり, 学習で見た種別を固める代わりに雛形外を削った。

2 周目: 素のモデルから 論文 (Soylu ら, EMNLP 2024) は素のモデルからプロンプトを先に最適化している。同じ順で素の Gemma 4 12B から GEPA を回すと, 7 案が全評価に進み, 1 案が採用された (検証平均 0.260→0.272)。この指示文は大規模問題の学習用・検証用だけで進化させたものだが, 素の Gemma に付けると, 使っていない雛形テストで正解が 345→364 に増えた (問題単位で 49 対 30, 符号検定 $p \approx 0.04$)。素のモデルではプロンプト最適化が効く。

次に, この指示文を system にして同じ教師データで学習し直したモデル (SB1) を, 既定の指示文で学習した S0 と比べた (表 6)。学習で見た種別ではわずかに上回るが, 問題単位の入れ替わりは大規模テストで 6 対 5, 元問題で 2 対 0 と小さく, 有意ではない。雛形外は 2 問のままだった。素のモデルで +19 問だった指示文の効果は, 学習後にはほぼ消える。学習データが同じなら, 学習時の指示文の文面は結果をほとんど変えない。

表 6: 指示文を先に最適化してから学習した効果 (正解数)

モデル	学習時の指示文	雛形テスト 445	大規模テスト 120	大規模元問題 28	雛形外 11
S0	既定	442	88 (参照以上 74)	12	2
SB1	GEPA で進化	445	89 (参照以上 78)	14	2

SB1 で 2 回目の GEPA を回しても (8 時間 16 分), 全評価に進んだ 1 案は元より低く (1.20 対 1.44), 採用 0 で指示文は変わらなかった。出発点を素のモデルにしても, 学習後のモデルではプロンプトが動かない点は同じである。

2 周を通して言えるのは次の二点である． GEPA が効くのは素のモデルに対してだけで， SFT 後は指示文を 1 文字も変えられない． そして， GEPA で得た指示文を SFT に持ち込んでも， 教師データが同じなら結果は変わらない． この問題集と教師データ量では， 「プロンプト最適化と fine-tuning の相乗効果」は観測できていない．

2.6 プロンプトの Qwen3.6 への転用

素の Gemma 向けに進化させた指示文が他のモデルにも効くかを， Qwen3.6 27B（思考あり）の大規模テスト 120 問で測った（表 7）． 既定の指示文では 4 問正解（平均 0.092），進化させた指示文では 9 問正解（平均 0.093）だった． 正解は倍以上に増えたが，問題単位では 8 対 3 で有意ではない（符号検定 $p \approx 0.23$ ）． 平均スコアが動かないのは，指示文が長くなった分だけ思考が延び，出力枠 65k を使い切ってコードを出せない問題が 8 から 18 に増えたためである． Gemma 向けに進化させた指示文は Qwen3.6 でも正解を増やす方向に働くが，Qwen の思考の長さとの兼ね合いがある．

表 7: Qwen3.6 27B の大規模テスト 120 問

指示文	正解	参照以上	平均スコア	出力枠切れ
既定（1,784 字）	4	1	0.092	8
GEPA で進化（4,152 字）	9	3	0.093	18

2.7 教師候補: GLM-5.3-Flash

前回の候補 GLM-5.3-Flash（320B，活性 18B）を llama.cpp で H200 2 枚と RAM に載せて試した． 生成は約 14 token/秒で，大規模問題 3 問のうち 2 問は思考だけで出力枠 32k を使い切り，残る 1 問も参照より約 10%悪かった． 1 問 40 分かかって正解 0 で，学習データ集めには使わないと判断した．

2.8 今後の方針: 交互最適化の再検証

学習済みモデルで GEPA が 1 案も採用できなかった原因は，「学習時の指示文への張り付き」か「教師データが GEPA の効果を埋めている」のどちらかである． これを切り分けるため，条件を変えて 2 本やり直す．

案 1: 指示文を複数混ぜて学習する． いまの学習は，1 万 6 千件の学習データすべての先頭に，同じ指示文（system）を 1 本だけ付けている． モデルはその 1 本を文面ごと覚えてしまい，少しでも違う指示文を渡すと崩れる． そこで，内容は同じで書き方だけ違う指示文を 5 本用意し（既定の compact，GEPA が進化させた Ib1，大規模向けの seed_hard，言い換え 2 本），学習データの行ごとにどれか 1 本を付けて学習する． 文面ではなく中身を覚えさせ，学習後に GEPA が指示文を書き換えても壊れないようにする狙いである． 判定は，学習後に 5 本どの指示文でも正解数がほぼ同じになること（張り付きが弱まった証拠）と，学習後の GEPA で 1 案以上が採用されること．

案 3: 教師データを使わず，自分の正解だけで学習する． 論文（Soylu ら）の条件に合わせ，Claude や Qwen の正解コードを使わない． 素の Gemma に GEPA の指示文を付けて

学習用の問題を解かせ、正解したものだけ（雛形で2,800対前後、大規模で6対の見込み）で学習し、その後に GEPA を回す。精度は落ちる前提の対照実験で、「教師データがあるときだけ GEPA の効果が消える」のかを確かめる。判定は、学習後に「素の Gemma + GEPA の指示文」（雛形 364 問）を上回り、学習後の GEPA で 1 案以上が採用されること。

どちらも採用 0 なら、「交互最適化は、学習時の指示文の固定化によって失敗する」という知見として 3 月に出す。手順と所要は OPERATIONS.md 15 章にある（案 1 が GPU 1 枚で約 1 日、案 3 が約 17 時間）。

3 今週の予定

対象を学習した 20 種別に絞り、Gemma 4 12B をその専用エンジンとして仕上げる方向で進める。

- 20 種別すべてを含む新データ（645 対）で Gemma 4 12B を学習し直し、表 4 と同じ 4 集合で測る
- 「生成 → 実行 → 検証 → 修復」の経路を Gemma 4 に組み込み、出力枠 16k で実行時エラーがどれだけ減るかを測る。プロンプト最適化は修復段の指示文に限って使う
- 対応外の要件を判定して弾く（または素のモデルに回す）経路を作る
- 交互最適化の再検証を 2 本流す（案 1: 指示文を複数混ぜて学習 → GEPA, 案 3: 自己生成データだけで学習 → GEPA）